

An Allocator-Aware `inplace_vector`

Document #: P3160R2
Date: 2024-10-14 20:04 EDT
Project: Programming Language C++
Audience: LEWG
Reply-to: Pablo Halpern
<phalpern@halpernwrightsoftware.com>
Arthur O'Dwyer
<arthur.j.odwyer@gmail.com>

Contents

- 1 Abstract
- 2 History
- 3 Motivation
 - 3.1 Some short examples
 - 3.2 General Motivation for Allocator-Aware Types
- 4 Proposal Summary
 - 4.1 Adding allocator support to `inplace_vector`
 - 4.2 Making `allocator` Freestanding
- 5 Design Decisions for Discussion
 - 5.1 *Uses-allocator construction* vs. `Allocator::construct/destroy`
 - 5.2 Behavior of move assignment and swap
- 6 Alternatives Considered
 - 6.1 Using `void` instead of `std::allocator<T>` for the default allocator
 - 6.2 `basic_inplace_vector<class T, size_t N, class Alloc = std::allocator<T>>`
- 7 Performance and Compile-Time Costs
- 8 Implementation Experience
- 9 Wording
 - 9.1 Feature-test macros
 - 9.2 Changes to `std::allocator`
 - 9.3 Changes to Allocator-aware Container Requirements
 - 9.4 Changes to `inplace_vector`
 - 9.4.1 Changes to Header Synopsis (24.3.7 [`inplace.vector.syn`])
 - 9.4.2 Changes to Overview (24.3.14.1 [`inplace.vector.overview`])

9.4.3 Changes to Constructors and Assignment Operators (24.3.14.2 [inplace.vector.cons])

9.4.4 Changes to `inplace_vector::swap` (in 24.3.14.5 [inplace.vector.modifiers])

9.4.5 Changes to Erasure (24.3.14.6 [inplace.vector.erase])

10 Acknowledgments

11 References

§ 1 Abstract

The `inplace_vector` proposal, [P0843R14], was accepted into the working paper without allocator support. We propose that `inplace_vector` *should* have allocator support and explores the pros and cons of adding such support directly into `inplace_vector` vs. into a separate `basic_inplace_vector` class template. In addition to providing an updated interface for `inplace_vector`, this paper proposes that `std::allocator` should be freestanding, so as allow its use as the default allocator type, consistent with other containers.

§ 2 History

New Information

LEWG discussed this paper in March 2024 at the Tokyo WG21 meeting. The room was roughly evenly split as to whether to pursue this paper further. SG14 (Low-latency Study Group) discussed this paper in their June 12, 2024 telecon to gage support for the use an allocator-aware `inplace_vector` in low-latency applications. A number of attendees voiced support for being able to have fine-grained memory control for allocator-aware objects stored within an `inplace_vector` whether or not they would use such a facility; most of the attendees did not object to adding allocator support to `inplace_vector`, provided it did not compromise its being freestanding. One poll was taken:

SG14 Poll: If the embeded issues can be solved, would you be OK with adding the allocator template parameter to `inplace vector`?

SF	F	N	A	SA
6	8	1	3	0

R2

- In response to SG14 feedback, `std::allocator` is proposed to be freestanding so that `inplace_vector<T, allocator<T>>` could be freestanding.
- Rebased to July 2024 working paper instead of P0843, now that [P0843R14] was accepted in the June, 2024 meeting in St. Louis.
- Removed much of the motivation, exploratory descriptions, and alternative designs. If an allocator-aware `inplace_vector` is to be part of the standard, only one or two designs make sense. Older design discussions can be found in the previous revisions of this paper.

R1

- [Proposal Summary](#) section added

- *Design Options* section renamed to [Alternatives Considered](#)
- *Compile-Time Data* section significantly expanded and renamed to [Performance and Compile-Time Costs](#)
- Miscellaneous copy editing

R0

- Initial version

§ 3 Motivation

§ 3.1 Some short examples

Consider an `inplace_vector` containing at most 10 short strings:

```
std::inplace_vector<std::string, 10> vec;

vec.emplace_back("hello");
vec.emplace_back("bye");
```

The code above would not allocate memory in most implementations because the strings stored in the `inplace_vector` fit within the small-string optimization. This non-allocating container is a key motivation for `inplace_vector`. But what if, every once in a while, a larger string needs to be stored in the vector?

```
vec.emplace_back("A longer string that does not fit in the SSO buffer");
```

To handle this possibility without accessing the heap, the programmer switches to `pmr::string` and sets aside a buffer that can hold the contents of larger strings:

```
char buffer[1024];
std::pmr::monotonic_buffer_resource mr(buffer, 1024);
std::inplace_vector<std::pmr::string, 10> vec;

vec.emplace_back("hello", &mr);
vec.emplace_back("bye", &mr);
vec.emplace_back("A longer string that does not fit in the SSO buffer", &mr);
```

Again, no heap accesses occur, but the code is brittle; it lacks abstraction by exposing the allocator at every `push_back` instead of relying on the usual invariant that all elements of the vector have the same allocator.

Consider what would happen if this code were in a generic function, where `vec` is a template parameter. The memory resource would not likely be available, nor could the generic code easily determine, for a given type of `vec`, whether an allocator should be passed to `push_back`. For example, if `vec` were created by `make_obj_using_allocator`, the result would likely not be what is expected:

```
pmr::polymorphic_allocator<> alloc{ &mr };
using V = std::inplace_vector<std::pmr::string, 10>;
auto vec = std::make_obj_using_allocator<V>(alloc, { "hello", "bye" });
assert(vec[0].get_allocator() == alloc); // FAILS
```

Even though an allocator is supplied, it is not used to construct the `pmr::string` objects within the resulting `inplace_vector` object because `inplace_vector` does not have the necessary hooks for

`make_obj_using_allocator` to recognize it as being allocator aware. Note that, although this example and the ones that follow use `pmr::polymorphic_allocator`, the same issues would apply to any scoped allocator.

A similar problem occurs if `inplace_vector` is used within an allocator-aware container:

```
std::pmr::vector<V> vo(alloc);  
  
vo.emplace_back({ "hello" });  
assert(vo.back()[0]->get_allocator() == alloc); // FAILS
```

Again, `inplace_vector` lacks the hooks needed to maintain the invariant that all parts of the outer vector use the same allocator.

§ 3.2 General Motivation for Allocator-Aware Types

Note: The text below is borrowed nearly verbatim from [P3002R1], which proposes a general policy for when types should use allocators.

In short, four principles underlie this policy proposal.

1. **The Standard Library should be general and flexible.** The user of a library class should have control, to the greatest extent possible, over how memory is allocated.
2. **The Standard Library should be consistent.** The use of allocators should be consistent with the existing allocator-aware classes and class templates, especially those in the containers library.
3. **The parts of the Standard Library should work together.** If one part of the library gives the user control over memory allocation but another part does not, then the second part undermines the utility of the first.
4. **The Standard Library should encapsulate complexity.** The generic application of allocators with maximum flexibility is potentially complex and is best left to the experts implementing the Standard Library. Users can choose their own subset of desirable allocator behavior only if the underlying Library classes allow them to choose their preferred approach, whether it be stateless allocators, statically typed allocators, polymorphic allocators, or no allocators.

§ 4 Proposal Summary

The proposal offered here includes changes to `inplace_vector` (24.3.14 [inplace.vector]¹), adding allocator support.

Because `inplace_vector` is intended to be available in a freestanding implementation, any use of `std::allocator` by `inplace_vector` must also be available in a freestanding implementation. Thus, there are changes proposed to `allocator` (20.2.10 [default.allocator]), making it freestanding, but see [Alternatives Considered](#) and [P3295R1] for other ways to tackle this issue.

§ 4.1 Adding allocator support to `inplace_vector`

This paper makes `inplace_vector` an *allocator-aware container* as described in 24.2.2.5 [container.alloc.reqmts] and as specified below.

- The class template would have a third template argument, `Allocator`, which would default to `std::allocator<T>`. The `std::pmr` namespace would also contain an alias:

```
template <class T, size_t N, class Allocator = allocator<T>>
class inplace_vector; // partially freestanding

namespace pmr
{
    template <class T, size_t N>
    using inplace_vector = std::inplace_vector<T, N,
        polymorphic_allocator<T>>;
}
```

- The interface would add `allocator_type` and `get_allocator` members:

```
using allocator_type = Allocator;

constexpr allocator_type get_allocator() const;
```

- Every constructor would have either an overload or a default argument allowing the allocator to be passed as its last argument:

```
constexpr inplace_vector() noexcept;
constexpr explicit inplace_vector(const allocator_type& a) noexcept;
constexpr explicit inplace_vector(size_type n);
    // freestanding-deleted
constexpr inplace_vector(size_type n, const allocator_type& a);
    // freestanding-deleted
constexpr inplace_vector(size_type n, const T& value, const
    allocator_type& a = {}); // freestanding-deleted
// etc..
```

- Elements of `inplace_vector<T, A>` are constructed via *uses-allocator construction with alloc* (20.2.8.2 [allocator.uses.construction]), where `alloc` is the return value of `get_allocator()`. This definition means that if `T` is an allocator-aware type using allocator type `A`, then `alloc` is used to construct each element as it is inserted into the `inplace_vector`. This is not the only possible design, however; see [Design Decisions for Discussion](#), below.

§ 4.2 Making allocator Freestanding

The `inplace_vector` class template is partially freestanding. Using `allocator<T>` as the default allocator for `inplace_vector` presents a problem, then, unless `allocator` is also (fully or partially) freestanding. The parts of `allocator` that cannot be freestanding are the `allocate` and `deallocate` member functions, since they allocate memory from a runtime heap that is not guaranteed to exist in freestanding implementations, and since `allocate` may throw.

Fortunately, `inplace_vector` never calls `allocate` or `deallocate`, and `allocator_traits<allocator<T>>::construct` and `allocator_traits<allocator<T>>::destroy`, which *are* called, are *already* freestanding. Theoretically, then, `allocator` could be *partially freestanding*, with `allocate` and `deallocate` being *freestanding-deleted*, but then, `std::allocator<T>` wouldn't meet the requirements of an allocator anymore. Rather than modify those requirements, we declare `allocate` and `deallocate` as `constexpr` for freestanding implementations. That way, they are available for `constexpr` applications and retain conformance with the allocator requirements, but are not available at runtime in the freestanding environment.

The changes proposed here are harmonious with Ben Craig's [P3295R1], which proposes that a number of standard library facilities, including the default allocator (`std::allocator`) be available in a freestanding implementation, but only in a `constexpr` context.

§ 5 Design Decisions for Discussion

§ 5.1 *Uses-allocator construction* vs. `Allocator::construct/destroy`

For wrapper types such as `tuple<T>`, an allocator passed to the constructor is passed through to the wrapped `T` object via *uses-allocator construction* (20.2.8.2 [allocator.uses.construction]), regardless of whether the allocator is a scoped allocator. The rationale for this design is that, since the `tuple` does not itself allocate memory, passing in an allocator that is compatible with `T` but which is not passed to the wrapped `T` object makes no sense. The same logic applies to the `basic_optional` and `basic_variant` templates proposed in [P2047R7] and [P3153R0], respectively.

On the other hand, the requirements on an allocator-aware container in 24.2.2.5 [container.alloc.reqmts] indicate that elements should always be constructed using `allocator_traits<Allocator>::construct` and destroyed using `allocator_traits<Allocator>::destroy`. A nonintuitive downside of following this convention for `inplace_vector` is that an allocator not having a special `construct` method would effectively be ignored (but might take up space in the object footprint). This would be the case for most non-scoped allocators:

```
template <class T>
struct NonScopedAlloc
{
    using value_type = T;

    NonScopedAlloc(...);

    value_type* allocate(std::size_t);
    void deallocate(value_type*, std::size_t);

    // `construct` and `destroy` are not declared
};

using MyString = std::string<char, std::char_traits<char>, NonScopedAlloc<char>>;

void f1()
{
    NonScopedAlloc a{ ... };
    inplace_vector<MyString, NonScopedAlloc<MyString>> v(a, { "x", "y" });
    assert(v[0].get_allocator() != a); // Allocator `a` was not used
}
```

Despite this unintuitive behavior, the main benefits of using `construct` and `destroy` is that the existing wording in 20.2.8.2 [allocator.uses.construction] applies unchanged, including the definitions of `Cpp17DefaultInsertable`, `Cpp17MoveInsertable`, `Cpp17CopyInsertable`, `Cpp17EmitConstructible`, and `Cpp17Erasable`. Moreover, `inplace_vector` could, in most cases, be used as a drop-in replacement for `vector`, or vice-versa, as needs change.

For most scoped allocators, including `pmr::polymorphic_allocator` and `scoped_allocator_adaptor<A>`, the two designs are equivalent. If a scoped allocator uses different

allocation mechanisms at different nesting levels (e.g., `scoped_allocator_adaptor<A1, A2>`), however *uses-allocator construction* will pass A1 to T's constructor (ignoring A2) whereas `construct` will pass either A1 or A2 to T, depending on the definition of `A1::construct`. It is possible to make either design behave more-or-less like the other by employing `scoped_allocator_adaptor<Allocator>`.

The choice between these two designs also affects the behavior of `swap`, as discussed next.

§ 5.2 Behavior of move assignment and swap

For all the other allocator-aware containers, move assignment and swap do not touch the elements of the containers; they simply change ownership of them. This behavior is not available for `inplace_vector`, as it's not possible to transfer ownership of elements that are not on the heap. There is a known issue with the absence of a swap specification in `inplace_vector` (see [LWG4151](#)), but there are potentially new issues with move assignment and swap in allocator-aware `inplace_vector`.

For two `inplace_vectors`, `a` and `b`, A typical implementation of `a.swap(b)` would be

1. *Possibly* swap their allocators (see below).
2. Perform an element-wise swap of the first n elements of `a` and `b`, where n is the smaller of `a.size()` and `b.size()`.
3. Move-construct the remaining elements from the larger `inplace_vector` to the smaller one.
4. Destroy the moved-from elements.
5. Adjust the size of each container (so that their initial sizes are swapped).

For an `Allocator` type for which `propagate_on_container_swap` (POCS) is `false_type`, there are no new issues; the allocators *are not swapped* and the precondition on swap devolves to the preconditions on the T's swap as well as the preconditions on move construction mediated by the allocator.

For an `Allocator` type for which POCS is `true_type`, the allocators *are swapped*, so if `a.construct` is used to construct elements of `inplace_vector`, `x`, then when `a.destroy` is eventually called on those elements, the value of `a` might have changed via swap (or via an assignment, if POCMA and/or POCCA are `true_type`). For the vast majority of allocators, this apparent mismatch will make no difference, but it is possible to create a situation where it is UB if, for example, `construct` and `destroy` track the address of their arguments.

There are a number of design options that could resolve this question:

1. Choose the *uses-allocator construction* design rather than `Allocator::construct/destroy` design, as described [above](#) to construct new elements, thus eliminating the opportunity for a mismatch. The simplification of the swap specification by itself, however, is not sufficient reason to make this design choice, as the issue is likely to come up again in the context of `small_vector` (or whatever it is eventually called), which will almost certainly use `construct/destroy`.
2. Leave the problem to the allocator. If an allocator provides both `construct` and `destroy`, and if each call to the latter must be matched up with a call to the former on the same object, and if that allocator defines POCS to `true_type`, then it is up to the allocator author to document this

fact and let users know not to swap two `inplace_vector` instances using different values of that allocator. This is Pablo's preferred resolution for the construct/destroy design.

3. If POCS is true, make it a precondition that the allocators of `a` and `b` compare equal. This is Arthur's preferred resolution. The benefit of this approach is that it is easy to test, within `inplace_vector` itself, that the precondition holds (whereas the previous approach pushes that concern to the allocator). The disadvantage is that POCS is often chosen *specifically* to enable swapping containers with unequal allocators, an intention that would be thwarted by this resolution.
4. Add new traits and/or member functions to `allocator_traits`. For example, an `allocator_traits::swap_elements` could call an `Allocator::swap_elements` that correctly patches up the allocator's data structure for the mismatched construct/destroy pair or an `allow_mismatched_destroy` trait could be added to determine whether the equal-allocators precondition must hold (where the trait would default to `true` if the allocator does not provide its own destroy member). This design space has not been explored and is not proposed here.

The wording in this paper uses resolution 1 for this revision.

§ 6 Alternatives Considered

Several alternative designs for an allocator-aware `inplace_vector` have been considered and discarded; see [P3160R1]. Only the ones still in play are described here.

§ 6.1 Using `void` instead of `std::allocator<T>` for the default allocator

This design would have two declarations:

```
template <class T, size_t N, class Allocator = void> class inplace_vector;  
template <class T, size_t N> class inplace_vector<void>;
```

The partial specialization for the `void` allocator would be identical (except for the extra template parameter) to the `inplace_vector` currently in the WP. It would not be an allocator-aware type; i.e., it would not have `allocator_type` or `get_allocator()` members. For non-`void` allocators, the full interface described in this paper would be available.

Pros:

- This alternative can be added to the WP without making `std::allocator` a freestanding type.
- The use of partial specialization would effectively force implementers to optimize for the default (non-allocator) case.

Cons

- The `void` default is different from all other standard containers, and requires special-case descriptions.
- The `void` specialization is not allocator-aware, which can be good or bad, but is, again, different from other standard containers.

§ 6.2 `basic_inplace_vector<class T, size_t N, class Alloc = std::allocator<T>>`

This approach defines a separate template, `basic_inplace_vector`, that is the same as the proposed allocator-aware `inplace_vector` but without affecting the interface of `inplace_vector` currently in the WP.

Pros:

- Reduces complexity of the non-allocator-aware `inplace_vector` specification and implementation.
- Can be added after C++26 without breaking ABI.

Cons:

- `inplace_vector` and `basic_inplace_vector` are separate, incompatible, types, increasing the *overall* specification and implementation complexity of the standard library.
- Some generic code will work with one and not the other. The user needs to make a decision, especially in generic code, regarding whether they ever expect to have allocator-aware elements. Choosing the shorter name is tempting, but comes at the expense of breaking scoped allocation in generic code.
- This approach is inconsistent with all other standard-library containers.
- Has all of the flaws of `inplace_vector<T, S, void>`, with none of the advantages.

§ 7 Performance and Compile-Time Costs

Experiments have shown that the proposed interface can be implemented with no runtime cost and negligible compile-time costs when allocators are not used. See [P3160R1].

§ 8 Implementation Experience

Pablo Halpern’s implementation of this proposal is available at https://github.com/phalpern/WG21-halpern/tree/main/P3160-AA-inplace_vector.

Arthur O’Dwyer’s implementation using the `construct/destroy` design is available at https://github.com/Quuxplusone/SG14/blob/master/include/sg14/aa_inplace_vector.h

Neither implementation includes the freestanding changes to `allocator`.

§ 9 Wording

All wording is relative to the July 2024 working paper, [N4986].

construct/destroy Wording: The wording below assumes the *uses-allocator* design. Wording changes needed for the `construct/destroy` design are called out in **construct/destroy Wording** boxes like this one. Where the authors disagreed, both potential wording changes are presented.

§ 9.1 Feature-test macros

In 17.3.2 [version.syn], add a feature test macro for freestanding allocator. The date, **20XXXXL**, should be replaced by the year and month that each feature was adopted.

```
#define __cpp_lib_allocator 20XXXXL           // also in <memory>

#define __cpp_lib_freestanding_allocator 20XXXXL // freestanding, also in
<memory>
```

The first macro indicates that the entirety of the allocator interface is supported, whereas the second macro indicates that *at least* all of the freestanding features are supported. In this case the minimal freestanding features are that `allocate` and `deallocate` are available in a `constexpr` context whereas the full feature set includes runtime support.

Also, update the version date for `inplace_vector`

```
#define __cpp_lib_inplace_vector 202406L20XXXXL // also in
<inplace_vector>
```

There is an existing issue whereby there was no feature-test macro for the minimal freestanding subset of `inplace_vector` (issue number TBD). The following would correct this issue. Note that if the issue is corrected prior to adopting this paper, that the date value should still be updated:

```
#define __cpp_lib_freestanding_inplace_vector 20XXXXL // freestanding,
also in <inplace_vector>
```

§ 9.2 Changes to `std::allocator`

Make the following addition to 20.2.2 [memory.syn].

```
// 20.2.10, the default allocator
template<class T> class allocator; // partially freestanding
template<class T, class U>
    constexpr bool operator==(const allocator<T>&, const allocator<U>&)
noexcept; // freestanding
```

Make the following additions to 20.2.10 [default.allocator].

```
namespace std {
    template<class T> class allocator {
    public:
        using value_type           = T;

        using size_type             = size_t;
        using difference_type       = ptrdiff_t;
        using propagate_on_container_move_assignment = true_type;

        constexpr allocator() noexcept;
        constexpr allocator(const allocator&) noexcept;
        template<class U> constexpr allocator(const allocator<U>&) noexcept;
        constexpr ~allocator();
        constexpr allocator& operator=(const allocator&) = default;
        [[nodiscard]] constexpr T* allocate(size_t n); // Note 1
```

```

[[nodiscard]] constexpr allocation_result<T*> allocate_at_least(size_t n);
// Note 1
constexpr void deallocate(T* p, size_t n); // Note 1
};
}

```

Note 1: For a freestanding implementation, it is implementation-defined whether `allocate`, `allocate_at_least`, and `deallocate` are `constexpr` rather than `consteval`.

§ 9.3 Changes to Allocator-aware Container Requirements

Make the following changes to 24.2.2.5 [container.alloc.reqmts].

Except for array ~~`and_inplace_vector`~~, all of the containers defined in Clause 24, 19.6.4, 23.4.3, and 32.9 meet the additional requirements of an *allocator-aware container*, as described below.

Given an allocator type `A` and given a container type `X` having a `value_type` identical to `T` and an `allocator_type` identical to `allocator_traits<A>::rebind_alloc<T>` and given an lvalue `m` of type `A`, a pointer `p` of type `T*`, an expression `v` that denotes an lvalue of type `T` or `const T` or an rvalue of type `const T`, and an rvalue `rv` of type `T`, the following terms are defined. If `X` is a specialization of `inplace_vector`, the terms below are defined as though `A` were `scoped_allocator_adaptor<allocator<T>, allocator_type>`. If `X` is not allocator-aware or is a specialization of `basic_string`, the terms below are defined as if `A` were `allocator<T>` — no allocator object needs to be created and user specializations of `allocator<T>` are not instantiated:

construct/destroy Wording: Omit the addition (“If `X` is a specialization...”), above, if the construct/destroy design is chosen.

In the same section (24.2.2.5 [container.alloc.reqmts]), also change *Note 2*:

[*Note 2:* A container calls `allocator_traits<A>::construct(m, p, args)` to construct an element at `p` using `args`, with `m == get_allocator()` (or `m == A(allocator<T>(), get_allocator())` in the case of `inplace_vector`). The default construct in `allocator` will call `::new((void*)p) T(args)`, but specialized allocators can choose a different definition. — *end note*]

construct/destroy Wording: Omit the changes to *Note 2* if the construct/destroy design is chosen.

In the same section, update the complexity clauses of move construction and swap for allocator-aware containers as follows:

`X u(rv);`

Postconditions: `u` has the same elements as `rv` had before this construction; the value of `u.get_allocator()` is the same as the value of `rv.get_allocator()` before this construction.

Complexity: ~~Constant~~ Linear for `inplace_vector`; constant for all other allocator-aware standard containers.

```
X u(rv, m);
```

Preconditions: T is *Cpp17MoveInsertable* into X.

Postconditions: u has the same elements, or copies of the elements, that rv had before this construction, `u.get_allocator() == m`.

Complexity: ~~Constant~~Linear for `inplace_vector`; constant for all other allocator-aware standard containers if `m == rv.get_allocator()`, otherwise linear.

```
a.swap(b)
```

Result: void

Effects: Exchanges the contents of a and b.

Complexity: ~~Constant~~Linear for `inplace_vector`; constant for all other allocator-aware standard containers.

§ 9.4 Changes to `inplace_vector`

§ 9.4.1 Changes to Header Synopsis (24.3.7 [`inplace.vector.syn`])

Update 24.3.7 [`inplace.vector.syn`] as follows.

24.3.7 Header `<inplace_vector>` synopsis [`inplace.vector.syn`]

construct/destroy Wording: For the construct/destroy design, replace all instances of *is-nothrow-ua-constructible-v* with *is-nothrow-allocator-constructible-v* (Pablo's preference) or *see below* (Arthur's preference) throughout the wording. See [Changes to Overview](#), below, for more detail on this design choice.

```
// mostly freestanding
#include <compare>           // see 17.11.1
#include <initializer_list> // see 17.10.2

namespace std {

    // exposition-only type traits
    template<class T, class A, class... X>
        constexpr bool is-nothrow-ua-constructible-v = see below; // exposition only

    // 24.3.14, class template inplace_vector
    template<class T, size_t N, class Allocator = allocator<T>>
        class inplace_vector; // partially freestanding

    // 24.3.14.6, erasure
    template<class T, size_t N, class A, class U>
        constexpr typename inplace_vector<T, N, A>::size_type
            erase(inplace_vector<T, N, A>& c, const U& value);
    template<class T, size_t N, class A, class Predicate>
```

```
constexpr typename inplace_vector<T, N, A>::size_type
    erase_if(inplace_vector<T, N, A>& c, Predicate pred);

namespace pmr {
    template<class T, size_t N>
        using inplace_vector = std::inplace_vector<T, N,
polymorphic_allocator<T>>;
}
}
```

§ 9.4.2 Changes to Overview (24.3.14.1 [\[inplace.vector.overview\]](#))

Update 24.3.14.1 [\[inplace.vector.overview\]](#) as follows:

An `inplace_vector` is a contiguous container. Its capacity is fixed and its elements are stored within the `inplace_vector` object itself. [Note: `inplace_vector` uses its allocator only to construct and destroy allocator-aware elements; it does not directly invoke the allocator's `allocate` or `deallocate` members. — end note]

An `inplace_vector` meets all of the requirements of a container (24.2.2.2), of a reversible container (24.2.2.3), of an allocator-aware container (24.2.2.5 [container.alloc.reqmts]), of a contiguous container, and of a sequence container, including most of the optional sequence container requirements (24.2.4). The exceptions are the `push_front`, `prepend_range`, `pop_front`, and `emplace_front` member functions, which are not provided. Descriptions are provided here only for operations on `inplace_vector` that are not described in one of these tables or for operations where there is additional semantic information.

For any `N` and `A`, `inplace_vector<T, N, A>::iterator` and `inplace_vector<T, N, A>::const_iterator` meet the `constexpr` iterator requirements.

For any `N > 0`, if `is_trivial_v<T>` is `false`, then no `inplace_vector<T, N, A>` member functions are usable in constant expressions.

Any member function of `inplace_vector<T, N, A>` that would cause the size to exceed `N` throws an exception of type `bad_alloc`.

For the purpose of this overview, a specialization of `std::allocator` is treated as being trivial, trivially default constructible, trivially copyable, trivially movable, and trivially destructible. [Note: for implementations having non-trivial special member functions in `allocator`, `inplace_vector` might not store a `allocator` object, but could use a new, value-initialized object each time a `allocator` object is required — end note]

Let `IV` denote a specialization of `inplace_vector<T, N, A>`. If `N` is zero and `is_empty_v<A>` is true, then `IV` is ~~both trivial and~~ empty. If `N` is zero and `is_trivial_v<A>` is true, then `IV` is trivial. Otherwise:

- If `is_trivially_copy_constructible_v<T>` && `is_trivially_copy_constructible_v<A>` is true, and if `A::select on container copy construction` does not exist, then `IV` has a trivial copy constructor.

- If `is_trivially_move_constructible_v<T> && is_trivially_copy_constructible v<A>` is `true`, then IV has a trivial move constructor.
- If `is_trivially_destructible_v<T>` is `true`, then:
 - If `is_trivially_destructible v<A>` is `true`, then IV has a trivial destructor.
 - If `is_trivially_copy_constructible_v<T> && is_trivially_copy_assignable_v<T> && is_trivially_copy_assignable v<A> && (allocator_traits<A>::propagate_on_container_copy_assignment::value || allocator_traits<A>::is_always_equal::value)` is `true`, then IV has a trivial copy assignment operator.
 - If `is_trivially_move_constructible_v<T> && is_trivially_move_assignable_v<T> && is_trivially_move_assignable v<A> && (allocator_traits<A>::propagate_on_container_move_assignment::value || allocator_traits<A>::is_always_equal::value)` is `true`, then IV has a trivial move assignment operator.

The exposition-only trait *is-nothrow-ua-constructible-v<T, A, X...>* is `true` if *uses_allocator construction* with allocator of type *A* and constructor arguments of types specified by *X...* (see 20.2.8.2 [\[allocator.uses.construction\]](#)) is known to be a non-throwing operation. [Note: This trait can be implemented by instantiating `is_nothrow_constructible_v<T, Y...>`, where *Y...* is the set of tuple arguments deduced by `uses_allocator_construction_args`. — end note]

construct/destroy Wording (Pablo’s preference): Replace the definition of *is-nothrow-ua-constructible-v*, above, with the definition of *is-nothrow-allocator-constructible-v* for the construct/destroy design:

The exposition-only trait *is-nothrow-allocator-constructible-v<T, A, X...>* is equivalent to `noexcept(A::construct(declval<A&>(), declval<T*>(), declval<X>()...))` if `A::construct` exists and `is_nothrow_constructible_v<T, X...>` otherwise. [Note: if `A::construct` does not exist, the allocator type is used to construct objects and is thus effectively ignored by `inplace_vector`. — end note]

construct/destroy Wording (Arthur’s preference): Remove the definition of *is-nothrow-ua-constructible-v*, above, entirely. Replace each use in an exception specification with *see below*. The exception specification will then be described per-operation for the case where `Allocator` is `std::allocator` and left unspecified otherwise.

For the move and allocator-extended move constructors, add:

Remarks: If `allocator_type` is `allocator<value_type>`, the exception specification is equivalent to `N == 0 ||`

`is_nothrow_move_constructible_v<value_type>`; otherwise, the exception specification is unspecified.

For the move-assignment operator, add:

Remarks: If `allocator_type` is `allocator<value_type>`, the exception specification is equivalent to `N == 0 || is_nothrow_move_assignable_v<value_type> && is_nothrow_move_constructible_v<value_type>`; otherwise, the exception specification is unspecified.

For the swap member function, add:

Remarks: If `allocator_type` is `allocator<value_type>`, the exception specification is equivalent to `N == 0 || is_nothrow_swappable_v<value_type> && is_nothrow_move_constructible_v<value_type>`; otherwise, the exception specification is unspecified.

Add an Allocator parameter to the definition of `inplace_vector`.

```
namespace std {  
    template<class T, size_t N, class Allocator = allocator<T>>  
    class inplace_vector {  
    public:  
        // types  
        using value_type      = T;  
        using allocator_type    = Allocator;  
        using pointer          = T*;  
        using const_pointer    = const T*;  
        using reference        = value_type&;  
        using const_reference  = const value_type&;  
        using size_type        = size_t;  
        using difference_type  = ptrdiff_t;
```

LWG Issue: *unlike other allocator-aware containers*, the `size_type`, `difference_type`, `pointer`, and `const_pointer` types are not aliases for the corresponding `allocator_traits` types. The `size_type` and `difference_type` types are specified by the standard to be types that can represent the distance between two iterators. Not being stored in allocated memory, the integral type that can represent the distance between elements in an `inplace_vector` is unrelated to the allocator. The meaning and intended use of `pointer` and `const_pointer` is not specified anywhere in the standard, so it is not clear whether it should be consistent with the allocator or consistent with the (raw) element storage. The definitions above are consistent with raw element storage, like `size_type` and `difference_type`, and is also consistent with the current use of `pointer` as the return type for `try_emplace_back` and `try_push_back`. Arthur believes that `pointer` should, instead be an alias for `allocator_traits::pointer`. If LWG decides that the intended meaning of `pointer` matches Arthur's understanding, then `try_emplace_back` and `try_push_back` should be changed to return `T*` instead of `pointer`. See [Arthur's blog post](#) for a good treatment of when fancy pointers are useful (though it doesn't argue one way or another on this particular issue).

And add allocator-aware constructors and get_allocator:

Drafting Note: The use of `type_identity_t<Allocator>` for the allocator-extended copy and move constructors is consistent with the other STL containers. CTAD needs it in order to handle allocator-argument deduction from `memory_resource*`, as in the case of `pmr::inplace_vector<X, 10> v;`
`pmr::unsynchronized_pool_resource mr; auto w = inplace_vector(v, &mr);`

```
// 24.3.14.2, construct/copy/destroy
constexpr inplace_vector() noexcept(noexcept(Allocator())) :
    inplace_vector(Allocator()) {};
explicit constexpr inplace_vector(const Allocator&) noexcept;
constexpr explicit inplace_vector(size_type n, const Allocator& =
    Allocator()); // freestanding-deleted
constexpr inplace_vector(size_type n, const T& value, const Allocator& =
    Allocator()); // freestanding-deleted
template<class InputIterator>
    constexpr inplace_vector(InputIterator first, InputIterator last,
        const Allocator& = Allocator()); // freestanding-
deleted
template<container-compatible-range<T> R>
    constexpr inplace_vector(from_range_t, R&& rg, const Allocator& =
    Allocator()); // freestanding-deleted
constexpr inplace_vector(const inplace_vector&);
constexpr inplace_vector(const inplace_vector&, const
    type_identity_t<Allocator>&);
constexpr inplace_vector(inplace_vector&&)
    noexcept(N == 0 || is_nothrow_move_constructible_v<T>)
    noexcept(N == 0 || is_nothrow_ua_constructible_v<T, Allocator, T&&>);
constexpr inplace_vector(inplace_vector&&, const
    type_identity_t<Allocator>&);
    noexcept(N == 0 || is_nothrow_ua_constructible_v<T, Allocator, T&&>);
constexpr inplace_vector(initializer_list<T> il, const Allocator& =
    Allocator()); // freestanding-deleted
constexpr ~inplace_vector();
constexpr inplace_vector& operator=(const inplace_vector& other);
constexpr inplace_vector& operator=(inplace_vector&& other)
    noexcept(N == 0 || (is_nothrow_move_assignable_v<T> &&
        is_nothrow_move_constructible_v<T>
        is_nothrow_ua_constructible_v<T, Allocator, T&&>));
constexpr inplace_vector& operator=(initializer_list<T>); // freestanding-
deleted
template<class InputIterator>
    constexpr void assign(InputIterator first, InputIterator last); //
freestanding-deleted
template<container-compatible-range<T> R>
    constexpr void assign_range(R&& rg); // freestanding-deleted
constexpr void assign(size_type n, const T& u); // freestanding-deleted
constexpr void assign(initializer_list<T> il); // freestanding-deleted
constexpr allocator_type get_allocator() const noexcept;
```

Also update the `noexcept` clause for the swap member function

```
constexpr void swap(inplace_vector& x)
    noexcept(N == 0 || (is_nothrow_swappable_v<T> &&
        is_nothrow_move_constructible_v<T>
        is_nothrow_move_constructible_v<T, Allocator, T&&>));
```

and the swap hidden-friend function.

```
constexpr friend void swap(inplace_vector& x, inplace_vector& y)
    noexcept(N == 0 || (is_nothrow_swappable_v<T> &&
        is_nothrow_move_constructible_v<T>
        is_nothrow_move_constructible_v<T>))
    noexcept(noexcept(x.swap(y)))
    { x.swap(y); }
```

§ 9.4.3 Changes to Constructors and Assignment Operators (24.3.14.2 [inplace.vector.cons])

Update the Constructors section as follows.

24.3.12.2 Constructors and assignment operators [inplace.vector.cons]

```
constexpr explicit inplace_vector(size_type n, const Allocator& = Allocator());
```

Preconditions: T is Cpp17DefaultInsertable into `*this`.

Effects: Constructs an `inplace_vector` with n default-inserted elements using the specified allocator.

Complexity: Linear in n.

```
constexpr inplace_vector(size_type n, const T& value, const Allocator& =
Allocator());
```

Preconditions: T is Cpp17CopyInsertable into `*this`.

Effects: Constructs an `inplace_vector` with n copies of value using the specified allocator.

Complexity: Linear in n.

```
template<class InputIterator>
    constexpr inplace_vector(InputIterator first, InputIterator last, const
Allocator& = Allocator());
```

Effects: Constructs an `inplace_vector` equal to the range [first, last) using the specified allocator.

Complexity: Linear in distance(first, last).

```
template<container-compatible-range <T> R>
    constexpr inplace_vector(from_range_t, R&& rg, const Allocator& = Allocator());
```

Effects: Constructs an `inplace_vector` object with the elements of the range `rg` using the specified allocator.

Complexity: Linear in `ranges::distance(rg)`.

```
constexpr inplace_vector& operator=(const inplace_vector& other);
```

Preconditions: `T` is *Cpp17CopyAssignable* and *Cpp17CopyInsertable* into `inplace_vector`.

Postconditions: `*this == other` is true.

Returns: `*this`.

Complexity: Linear

```
constexpr inplace_vector& operator=(inplace_vector&& other)
noexcept(N == 0 || (is_nothrow_move_assignable_v<T> &&
                    is_nothrow_ua_constructible_v<T, Allocator, T&&>));
```

Preconditions: `T` is *Cpp17MoveAssignable* and *Cpp17MoveInsertable* into `inplace_vector`.

Effects: All existing elements of `*this` are either move assigned to or destroyed.

Returns: `*this`.

Postconditions: If `*this` and `other` do not refer to the same object, `*this` is equal to the value that `other` had before this assignment.

Complexity: Linear

***uses-allocator* Wording:** The above definitions for the assignment operators can be removed for the *uses-allocator* design because they don't add much to the descriptions allocator-aware container requirements.

***construct/destroy* Wording:** This wording is for **resolution 2 (Pablo's preference)** described in the [Behavior of move assignment and swap](#) section, above. Add *Remarks* clauses for the assignment operators.

For the copy-assignment operator, add:

Remarks: If

`allocator_traits<allocator_type>::propagate_on_container_copy_assignment::value` is true, then, after the allocator is assigned, `allocator_traits<allocator_type>::destroy` might be invoked with an allocator having a different value than when `allocator_traits<allocator_type>::construct` was invoked on a some element. This construct-assign-destroy sequence is well defined for all allocator types in the standard library. [Note: No object is ever *deallocated* from an allocator having a different value than that from which it was *allocated*. — end note]

For the move-assignment operator, add:

Remarks: If

`allocator_traits<allocator_type>::propagate_on_container_move_assignment::value` is true, then, after the allocator is assigned, `allocator_traits<allocator_type>::destroy`

might be invoked with an allocator having a different value than when `allocator_traits<allocator_type>::construct` was invoked on a some element. This construct-assign-destroy sequence is well defined for all allocator types in the standard library. [Note: No object is ever *deallocated* from an allocator having a different value than that from which it was *allocated*. — end note]

Drafting note: The *Remarks* clauses, above, as well as the one for `swap`, below, are essentially the same. It might be better to centralize some or all of the text.

construct/destroy Wording: This wording is for **resolution 3 (Arthur's preference)** described in the [Behavior of move assignment and swap](#) section, above. Extend the *Preconditions* clauses for the assignment operators.

For the copy-assignment operator, modify the *Preconditions* as follows.

Preconditions: T is *Cpp17CopyAssignable* and *Cpp17CopyInsertable* into `inplace_vector`. If `allocator_traits<allocator_type>::propagate_on_container_copy_assignment::value` is true, then `other.get_allocator() == this->get_allocator()` is true.

For the move-assignment operator, modify the *Preconditions* as follows.

Preconditions: T is *Cpp17MoveAssignable* and *Cpp17MoveInsertable* into `inplace_vector`. If `allocator_traits<allocator_type>::propagate_on_container_move_assignment::value` is true, then `other.get_allocator() == this->get_allocator()` is true.

§ 9.4.4 Changes to `inplace_vector::swap` (in 24.3.14.5 [\[inplace.vector.modifiers\]](#))

Add the definition of `swap` to the end of 24.3.14.5 [\[inplace.vector.modifiers\]](#).

```
constexpr void swap(inplace_vector& x)
    noexcept(N == 0 || (is_nothrow_swappable_v<T> &&
        is_nothrow_ua_constructible_v<T, Allocator, T&&>));
```

Preconditions: Let M be $\min(\text{size}(), \text{x.size}())$. For each non-negative integer $n < M$, $(*this)[n]$ is swappable with $x[n]$ (16.4.4.3 [\[swappable.requirements\]](#)).

Effects: Exchanges the contents of $*this$ and x .

Complexity: Linear

construct/destroy Wording: This wording is for **resolution 2 (Pablo's preference)** described in the [Behavior of move assignment and swap](#) section, above: Add the following *Remarks* for the `construct/destroy` design.

Remarks: If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is true, then, after the allocators are swapped, `allocator_traits<allocator_type>::destroy` might be invoked with an allocator having a different value than when `allocator_traits<allocator_type>::construct` was invoked on a some

elements of both `inplace_vectors`. This construct-swap-destroy sequence is well defined for all allocator types in the standard library. [Note: No object is ever *deallocated* from an allocator having a different value than that from which it was *allocated*. — end note]

construct/destroy Wording: This wording is for **resolution 3 (Arthur’s preference)** described in the [Behavior of move assignment and swap](#) section, above: Change the *Preconditions* for swap for the construct/destroy design as follows.

Preconditions: Let M be `min(size(), x.size())`. For each non-negative integer $n < M$, `(*this)[n]` is swappable with `x[n]` (16.4.4.3 [\[swappable.requirements\]](#)). If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is true, then `other.get_allocator() == this->get_allocator()` is true.

§ 9.4.5 Changes to Erasure (24.3.14.6 [\[inplace.vector.erase\]](#))

Add the allocator parameter to the erase non-member functions in 24.3.14.6 [\[inplace.vector.erase\]](#):

24.3.14.6 Erasure [\[inplace.vector.erase\]](#)

```
template<class T, size_t N, class A, class U = T>
constexpr size_t erase(inplace_vector<T, N>& c, const U& value);
```

Effects: Equivalent to:

```
auto it = remove(c.begin(), c.end(), value);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

```
template<class T, size_t N, class A, class Predicate>
constexpr size_t erase_if(inplace_vector<T, N>& c, Predicate pred);
```

Effects: Equivalent to:

```
auto it = remove_if(c.begin(), c.end(), pred);
auto r = distance(it, c.end());
c.erase(it, c.end());
```

§ 10 Acknowledgments

The authors would like to thank each other for a congenial working relationship whereby issues could be raised and addressed, and disagreements presented fairly within this paper.

Thanks to SG14 for helping me identify the key requirements of `inplace_vector` in embedded environments.

Thanks to Ben Craig for his work on freestanding allocator, which I’ve excerpted in this paper.

§ 11 References

- [N4986] Thomas Köppe. 2024-07-16. Working Draft, Programming Languages — C++. <https://wg21.link/n4986>
- [P0843R14] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2024-06-26. `inplace_vector`. <https://wg21.link/p0843r14>
- [P2047R7] Nina Ranns, Pablo Halpern Ville Voutilainen. 2024-02-15. An allocator-aware optional type. <https://wg21.link/p2047r7>
- [P3002R1] Pablo Halpern. 2024-02-15. Policies for Using Allocators in New Library Classes. <https://wg21.link/p3002r1>
- [P3153R0] Nina Ranns, Pablo Halpern, Ville Voutilainen. 2024-02-15. An allocator-aware variant type. <https://wg21.link/p3153r0>
- [P3160R1] Pablo Halpern. 2024-03-09. An allocator-aware `'inplace_vector'`. <https://wg21.link/p3160r1>
- [P3295R1] Ben Craig. 2024-09-15. Freestanding constexpr containers and constexpr exception types. <https://wg21.link/p3295r1>
-

1. All citations to the Standard are to working draft N4986 unless otherwise specified. ↩